LeBlanc v3: Status, Research Direction, and MCP Deliverable

B.Tech Capstone Project Status Report

22 August 2026

Abstract

LeBlanc began as a system for improving the security of LLM-generated Python code through CWE-aware prompt enrichment, static analysis, and iterative repair. Preliminary team experiments suggest that recent models may receive little additional benefit from this older scaffolding. This is not a failed outcome: it motivates the sharper question, *when does legacy security scaffolding still add value, and when is it redundant?* This report records the implemented artefacts, distinguishes preliminary observations from reproducible evidence, sets revised research questions, and defines the Model Context Protocol (MCP) server as the practical project deliverable.

1 Project in Plain Terms

Earlier secure-code research found that LLMs could sometimes produce safer code if they were given explicit vulnerability advice before writing, or a security-tool report after writing. LeBlanc tests whether that still matters for newer models. It does not claim to have invented these techniques.

Prompt → security warning → LLM code → Bandit and Semgrep scan → optional repair → functional smoke test.

The central conclusion may be that scaffolding helps old models but not current models, helps only selected vulnerability categories, or produces repairs that look scanner-clean while breaking intended behaviour. Each is a meaningful result.

2 What Has Been Built

- **Engine A**: local CWE-aware prompt enrichment.
- **Engine B**: Bandit plus a pinned Semgrep ruleset, with normalised findings.
- **Engine C**: scanner-guided LLM repair, limited to three iterations.
- **Engine D**: sandboxed functional smoke tests for final code.
- A SQLite-backed batch experiment runner, Flask dashboard, metrics layer, and frozen dataset.
- An MCP server exposing prompt analysis, in-memory code scanning, path scanning, response auditing, optional repair, and local project-status tools.

The dataset has 50 Python prompts: 10 original realistic prompts and 40 SecurityEval prompts. Twenty prompts have executable smoke tests and reference solutions. The other 30 support static security analysis only because their interfaces or safe happy paths are underspecified.

3 Current Status

Area	Status
Implementation	Core pipeline, dataset, test harness, dashboard, batch runner, and MCP prototype exist.
Preliminary work	The team reports approximately 600 LLM calls/attempts across 20 testable prompts and three models. The qualitative observation is that recent models saw little incremental benefit from enrichment and repair.
Repository evidence	The checked-in SQLite database currently contains one stored run only. Preliminary results must be recovered, imported, or rerun before they can support paper claims.
Paper/analysis	No reproducible final figures, analysis notebook outputs, or paper results are stored yet.

The "600" number must be made precise. Twenty prompts × three models × three modes × three repetitions equals 540 primary experimental cells. Repair calls and retries can make the number of LLM API calls exceed that. The final database must distinguish primary runs, repair iterations, retries, and failed calls.

4 Why the Research Direction Changes

The original proposal presented LeBlanc as a broadly useful secure-code-generation system. That is too strong. SecuCoGen studied vulnerability-aware prompting; Guiding AI to Fix Its Own Flaws studied security hints and repair feedback; CodeSecEval evaluated secure generation and repair; and CODEGUARD+ showed why security must be measured with correctness [1, 2, 3, 4].

LeBlanc should instead be a **temporal re-evaluation**. It asks whether older scaffolding still adds measurable benefit as models improve. A conclusion of "little or no benefit for current models" does not disprove prior work. It says that earlier results were conditional on the models and period in which they were measured.

5 Revised Research Questions

1. **Baseline change:** How do old/small and current models differ in static vulnerability rate on identical tasks?
2. **Enrichment value:** What is the paired difference between plain generation and CWE-aware prompt enrichment for each model?
3. **Repair value:** Among initially vulnerable outputs, how often does scanner-guided repair reach a clean scanner verdict, and how quickly?
4. **Repair inflation:** Among scanner-clean repairs, how often do functional smoke tests fail? This measures "secure-looking but broken" code.
5. **Practical rule:** For which model and task combinations is the measured improvement large enough to justify invoking the MCP workflow?

6 Model Plan

The existing code lists Llama 3.1 8B, Llama 3.3 70B, GPT-4o mini, Gemini 2.5 Flash, Claude Haiku 4.5, and DeepSeek Chat. This is only a starting configuration: availability changes and the current design contains only one small/old baseline despite describing two in the documentation.

Before final collection, freeze a dated model manifest containing one or two old/small baselines, one or two 2024-era capable comparisons, and at least two current 2026 models from different providers. Record exact identifiers, access date, price, temperature, token limit, scanner version, and preflight result. Interpret generation carefully: model age is confounded with provider, size, training data, and API behaviour.

For every selected prompt/model pair, compare `plain`, `enriched`, and `enriched_repair` with matched repetitions. Report extraction failures, static vulnerability rate, repair convergence, functional pass rate, and the joint scanner-clean-and-functional rate.

7 MCP Deliverable

The MCP server is the usable B.Tech deliverable and a demonstration of the evaluated workflow; it is not proof that the workflow is effective. A coding agent can ask it to analyse a risky task, scan generated code, audit a workspace-scoped project, and optionally request a repair. Results should govern the product behaviour: if scaffolding helps only selected cases, invoke it selectively; if its benefit is negligible, retain it chiefly as a scanner-based audit tool.

Before broader use, path scanning must be restricted to approved workspace roots. A scanner-clean verdict must never be marketed as a security guarantee, and repair output must remain advisory until tests and human review pass.

8 Immediate Work

1. Recover or rerun the preliminary experiment data and document exactly what "600" means.
2. Archive database exports, raw outputs, scanner versions, and model preflight results.
3. Freeze the model fleet and run a documented pilot before final collection.
4. Audit static-analysis false positives and report the limited 20/50 functional-test coverage.
5. Generate every final figure and table from the archived database.
6. Remove the unrelated BreachTrace AWS CloudTrail sprint document from the LeBlanc research archive.

9 Focused Literature Review

The review should centre on SecuCoGen (CWE-aware prompting), Guiding AI to Fix Its Own Flaws (hints and repair feedback), CodeSecEval (secure generation/repair evaluation), and CODEGUARD+ (security plus correctness). HexaCoder and LPO/DiSCo are useful contrasts because they alter or fine-tune models, whereas LeBlanc is API-only scaffolding. The GitHub Copilot replication study supports re-measuring security behaviour as models change over time [5, 6, 7].

10 Conclusion

LeBlanc should proceed as an evidence-led re-evaluation, not as a claim to have invented secure code generation. Its contribution can be a practical answer to: *when is warning-and-repair security scaffolding still worth using, and when is it obsolete?* The MCP server provides a concrete system artifact whose role can be decided by the evidence.

References

[1] Wang et al. "Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on Vulnerability Mitigation." SecuCoGen, 2024. `docs/researchdocs/Two.pdf`

[2] Yan et al. "Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation." 2025. `docs/researchdocs/One.pdf`

[3] Wang et al. "Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval." `docs/researchdocs/Three.pdf`

[4] Fu et al. "Constrained Decoding for Secure Code Generation." CODEGUARD+. `docs/researchdocs/Five.pdf`

[5] Hajipour et al. "HexaCoder: Secure Code Generation via Oracle-Guided Synthetic Training Data." `docs/researchdocs/Seven.pdf`

[6] Hasan et al. "Teaching an Old LLM Secure Coding: Localized Preference Optimization on Distilled Preferences." `docs/researchdocs/Four.pdf`

[7] Majdinasab et al. "Assessing the Security of GitHub Copilot's Generated Code: A Targeted Replication Study." `docs/researchdocs/Eight.pdf`

[8] LeBlanc internal records: `README.md`; `docs/00_DEFINITION.md` through `docs/05_EXECUTION_PLAN.md`; `dataset/tests/MANIFEST.md`; and `backend/mcp_server.py`.